Drone Vision Challenges

Student Handout and Teacher Appendix

This handout uses prerecorded drone videos as a simulated UAV camera feed. Students practice OpenCV, NumPy, class design, telemetry parsing, target tracking, control, and mission logic in a sequence that mirrors a future ROS 2 architecture.

Course tools	Python, OpenCV, NumPy, class-based design
Dataset	Teacher-selected folder configured in video_config.py or DRONE_VIDEO_DIR
Project files	F:\dev\drone\activities\challenges
End goal	Build a miniature perception-control-mission pipeline from prerecorded flights

Sequence Overview

Teacher setup: choose the dataset folder once in video_config.py or set the DRONE_VIDEO_DIR environment variable before class.

- Challenge 1-2: camera feed and structured frame data
- Challenge 3: manual piloting commands and HUD
- Challenge 4: telemetry parsing from DJI subtitle files
- Challenge 5: target perception with OpenCV and NumPy
- Challenge 6: proportional control
- Challenge 7: mission state machine
- Challenge 8: full pipeline integration

Challenge 1: Camera Viewer Class

Goal: Build a class that opens a drone video, reads frames, and overlays frame number and timestamp.

Video: Bluemlisalphutte Flyover.mp4 from the teacher-selected video folder

Starter files: challenge_01_camera_student.py and challenge_01_camera_teacher.py

Student tasks

- Complete the DroneVideoPlayer class.
- Open the video with cv2.VideoCapture.
- Resize frames for classroom use.
- Draw frame and time overlays.
- Quit cleanly with the q key.

Success criteria

- The video plays smoothly.
- Overlay values update every frame.
- The program closes without errors.

Challenge 2: Frame Packet Class

Goal: Create a message-like dataclass that carries the frame plus NumPy-derived brightness statistics.

Video: Stockflue Flyaround.mp4 from the teacher-selected video folder

Starter files: challenge_02_packet_student.py and challenge_02_packet_teacher.py

Student tasks

- Validate the camera stream.
- Create and return FramePacket objects.
- Convert frames to grayscale with OpenCV.
- Compute average brightness using NumPy.
- Display the brightness on the image.

Success criteria

- Students use a dataclass to structure data.
- Brightness values change with the scene.
- The packet acts like a future ROS 2 message.

Challenge 3: Manual Pilot Class

Goal: Simulate manual flight control with keyboard commands for roll, pitch, yaw, and throttle.

Video: Bluemlisalphutte Flyover.mp4 from the teacher-selected video folder

Starter files: challenge_03_teleop_student.py and challenge_03_teleop_teacher.py

Student tasks

- Map keys to each command axis.
- Clamp the values to a safe interval.
- Add a reset on the space key.
- Draw the command HUD on the video.

Success criteria

- Each key updates the correct axis.
- Values stay in the safe range.
- Students can explain the difference between roll, pitch, yaw, and throttle.

Challenge 4: Telemetry Parser Class

Goal: Parse DJI subtitle telemetry and align the sensor data with the video timeline.

Video: DJI_0574.MP4 from the teacher-selected video folder

Starter files: challenge_04_telemetry_student.py and challenge_04_telemetry_teacher.py

Student tasks

- Open the matching SRT file.
- Parse time, GPS, altitude, and barometer values.
- Find the latest telemetry sample for the current video time.
- Overlay telemetry data on the video.

Success criteria

- Telemetry appears only when a matching sample exists.
- Values update with time as the video plays.
- Students see that drones use multiple sensing sources.

Challenge 5: Virtual Target Tracker Class

Goal: Add a virtual target to the frame, detect it with HSV thresholding, and compute its centroid with NumPy.

Video: DJI_0596.MP4 from the teacher-selected video folder

Starter files: challenge_05_target_student.py and challenge_05_target_teacher.py

Student tasks

- Draw a moving red target onto each frame.
- Convert the image to HSV.
- Threshold the red color range.
- Use NumPy to compute centroid and area.
- Visualize the target and mask.

Success criteria

- The mask isolates the target clearly.
- The centroid follows the target motion.
- Students connect vision to measurable target state.

Challenge 6: Proportional Controller Class

Goal: Convert perception error into roll, pitch, yaw, and throttle commands using a proportional controller.

Video: No live video required; uses example detections

Starter files: challenge_06_controller_student.py and challenge_06_controller_teacher.py

Student tasks

- Implement a clamp helper.
- Define behavior when the target is missing.
- Use negative feedback from image error.
- Add forward pitch only when the target appears far away.

Success criteria

- Commands point in the corrective direction.
- Outputs stay within limits.
- Students can explain why sign matters in feedback control.

Challenge 7: Mission Manager Class

Goal: Create a high-level mission state machine with SEARCH, ALIGN, HOLD, and LAND states.

Video: No live video required; uses target state sequence

Starter files: challenge_07_mission_student.py and challenge_07_mission_teacher.py

Student tasks

- Search by yawing when the target is missing.

- Align when the target is visible but off-center.
- Hold position when centered.
- Land after a stable hold period.

Success criteria

- The state progression is explainable and stable.
- Students separate mission logic from low-level control.
- The hold counter prevents immediate landing.

Challenge 8: Full Pipeline Integration

Goal: Integrate camera, target injection, detection, control, and HUD into one class-based mock autonomy pipeline.

Video: DJI_0790.MOV from the teacher-selected video folder

Starter files: challenge_08_integration_student.py and challenge_08_integration_teacher.py

Student tasks

- Add the virtual target.
- Detect it and compute normalized image error.
- Generate commands from the detection.
- Draw a complete HUD with target and command data.

Success criteria

- The final script behaves like a miniature autonomous drone stack.
- Students can trace data from sensing to action.
- The design is easy to map to future ROS 2 nodes.

Teacher Appendix

Use this appendix while circulating in class. It highlights what to check, what students often confuse, and which ideas are worth discussing before they move to the next challenge.

Challenge 1: Camera Viewer Class

Teacher prompts and checks

- This is the first camera-node style exercise.
- Watch for missing str(video_path) when opening the file.
- Ask students why FPS is needed for timestamps.

Challenge 2: Frame Packet Class

Teacher prompts and checks

- This introduces structured messages before ROS 2.
- A common mistake is forgetting to increment frame_index.
- Students should explain why np.mean is useful here.

Challenge 3: Manual Pilot Class

Teacher prompts and checks

- This works like a future cmd_vel publisher.
- Students often swap yaw and roll at first.
- Ask how they would stop the drone in an emergency.

Challenge 4: Telemetry Parser Class

Teacher prompts and checks

- This is a good place to discuss sensor fusion.
- Latitude and longitude order is a frequent source of confusion.
- Remind students not every clip has telemetry.

Challenge 5: Virtual Target Tracker Class

Teacher prompts and checks

- This is the perception stage of the pipeline.
- Students often try BGR thresholds first; steer them toward HSV.
- Make sure they handle the no-detection case.

Challenge 6: Proportional Controller Class

Teacher prompts and checks

- This challenge is ideal for debugging on paper first.
- A common error is positive feedback instead of negative feedback.
- Ask what happens if gains are too high.

Challenge 7: Mission Manager Class

Teacher prompts and checks

- This prepares students for behavior trees or state machines in ROS 2.
- Students often forget to reset the hold counter.
- Ask them which transitions are safety critical.

Challenge 8: Full Pipeline Integration

Teacher prompts and checks

- This is the capstone exercise of the sequence.
- Encourage students to identify what would become separate ROS 2 nodes later.
- Normalized error is the key concept to verify here.